David Guijosa

Software Engineer

Riverside, CA | (951) 941-8408 | davidgin641@gmail.com
linkedin.com/in/davidguijosa | github.com/Davidzent | zntsns.com

SUMMARY

Software Engineer with 4+ years shipping production systems end to end: React and TypeScript frontends over Java, Spring Boot and NestJS services on PostgreSQL, containerized with Docker and deployed on GCP. Sole developer on most of 12+ client engagements, from requirements through deploy. One system still in production at Riverside City College five years on.

TECHNICAL SKILLS

Languages: Java, TypeScript, JavaScript, SQL, PHP, C#
Backend: Spring Boot, Spring Security, Spring Data, Hibernate, REST APIs, Microservices, Apache Kafka, NestJS, Node.js (Express), Prisma, OAuth 2.0, JWT, Stripe Connect
Frontend & Mobile: React, React Native (Expo), Google Maps API, Angular, Vite, Tailwind CSS, HTML/CSS, AJAX
Databases: PostgreSQL, MySQL, SQLite, Redis, Elasticsearch
Cloud & DevOps: Google Cloud Platform, AWS (EC2, S3, Lambda, DynamoDB), Terraform, Docker, Jenkins, GitHub Actions, Maven, SonarQube
Testing & Practices: JUnit, Mockito, Testcontainers, Vitest, Jest, Postman, Git, Agile/Scrum, Code Review, CI/CD, AI-Assisted Development (GitHub Copilot, Claude, Codex), Idempotency Keys, Row-Level Locking

EXPERIENCE

Full-Stack Developer | Freelance **Jun 2023 – Present**

- Delivered full-stack applications for 12+ clients, from solo operators to a four-person product team: React or Angular with TypeScript over Java, Spring Boot and Node services on PostgreSQL. Sole developer on most engagements, and replaced manual releases with Jenkins, SonarQube and Docker pipelines on GCP that cut deployment cycles 30%.
- Own both backends and the Android client of a travel platform: a NestJS/PostgreSQL identity service (36 endpoints) and a booking and payments service (278 endpoints, 94 Prisma models, 47 migrations), plus an Expo/React Native app of ~505 TypeScript files.
- Designed the Stripe Connect money path end to end: idempotency-keyed PaymentIntents, a webhook-driven state machine that claims each transition under a row lock, partner transfers reversed automatically on refund, and a reconciliation job that diffs local records against Stripe.
- Diagnosed intermittent data loss on a client's Node.js service under concurrent load, traced it to unserialized writes, and eliminated it with row-level locking at the SQL layer.

Software Engineer | Revature, placed at Cognizant Technology Solutions **Jan 2022 – Apr 2023**

Enterprise training program (Jan – Apr 2022), then full-time client placement at Cognizant (Apr 2022 – Apr 2023).

- Developed three REST microservices in Java and Spring Boot covering authentication, authorization and core business logic for an internal employee platform spanning a social feed, time tracking and payroll. Implemented OAuth 2.0 with Spring Security, held the required 80% SonarQube coverage gate with zero critical vulnerabilities, and owned merges into main.
- Flagged an unowned storage and bandwidth cost, images served raw at up to 10 MB, and cut payloads to under 100 KB with WebP conversion and compression at the upload boundary.

Student Employee, Computer Lab & Instructional Support | Riverside City College **Jan 2019 – Jun 2021**

- Fixed a timing-out page in the lab attendance system by tracing it to front-end over-fetching and moving row limiting into the query, then added an Excel export that replaced end-of-semester compilation by hand. Still in production five years on, with credit on the live site.

PROJECTS

AwardTrace — Federal Contract Search Java 21 • Spring Boot 4 • Kafka • Elasticsearch • PostgreSQL • AWS • React 19
Live: awardtrace.zntsns.com | Code: github.com/Davidzent/AwardTrace

- Built a federal contract search platform on AWS that streams 54,063 USAspending transactions through Kafka into PostgreSQL and Elasticsearch, sustaining 1,068 transactions per second on a single c7i-flex.large node.
- Classified 31,344 award descriptions with Claude Haiku 4.5 through the Message Batches API for $3.25, and made its categories the site's default only after it scored 65.7% on 181 hand-labeled descriptions against a 59.7% baseline from government product codes.
- Checked the data model against real USAspending files before writing the parser, found three wrong assumptions (running totals, re-emitted modification dates, and deletes with no award key), and redesigned awards as projections recomputed from their transactions.

Warehouse — Inbound Receiving System **Java 17 • Spring Boot 4 • PostgreSQL • React 19**
Live: zntsns.com/warehouse | Code: github.com/Davidzent/Warehouse-API, Warehouse-UI

- Modeled inbound receiving over nine PostgreSQL tables with a 110% over-receipt tolerance, duplicate-line rejection and damaged-versus-usable stock separation, behind a JWT resource server with role checks.
- Metered the API with a three-tier token bucket allowing 5 sign-ins, 10 writes and 60 reads a minute per IP, and serialized receipts with pessimistic row locking. 52 JUnit tests, 11 against embedded PostgreSQL rather than mocks.

EDUCATION

Associate of Science, Computer Science | Riverside City College **2021**